



MODULO VERTICALE

UI Programming

Lezione 2



COSE DA SAPERE PURTROPPO O PER FORTUNA



```
1. FSlateApplication::Tick
2. FSlateApplication::TickAndDrawWidgets ...
3. SWidget::Paint
4. SWidget::OnPaint ...
5. SObjectWidget::Tick
6. UUserWidget::NativeTick (this is when Blueprint will receive tick too)
```

Tick

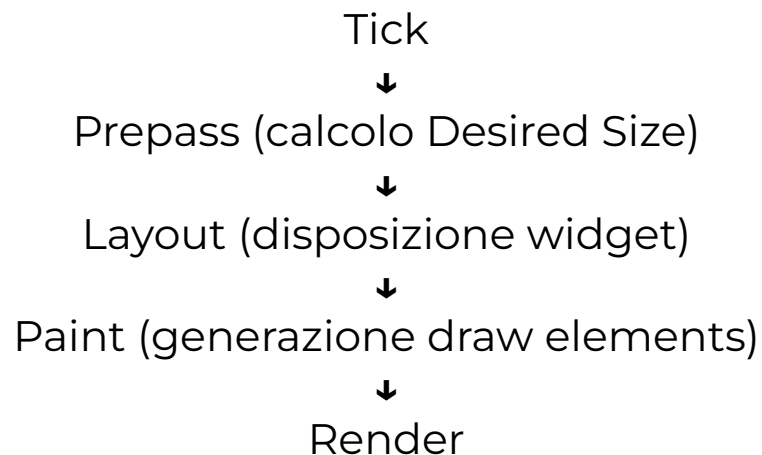
I widget non “tickano”, non hanno componenti per il tick e non hanno tick group.

L'ordine in cui si verifica il **tick** dei widget è durante il passaggio di Paint tramite le varie chiamate provenienti da **FSlateApplication**, in modo che la funzione tick di un widget venga chiamata **solo** quando è sullo schermo e viene renderizzato.

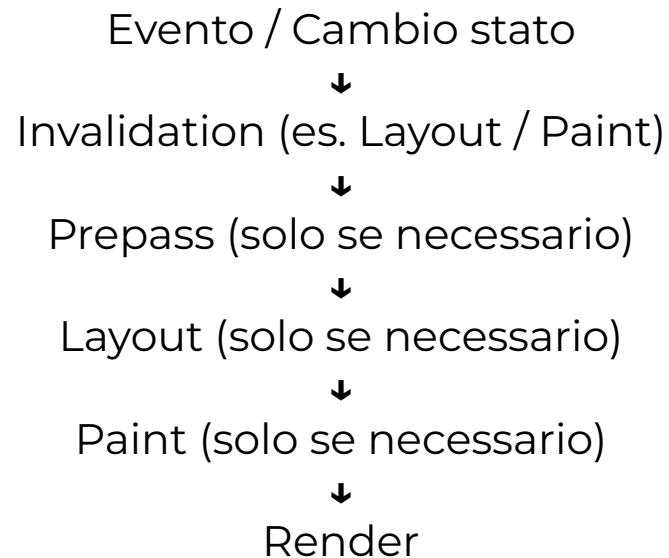


Pipeline di Rendering UI

Senza Invalidation (ogni frame)



Con Invalidation attiva



Nei frame successivi (nessun cambiamento)



Usa CACHE (salta tutta la pipeline)



Pipeline di Rendering UI

Prepass → Layout → Paint

- Prepass
 - Ricalcola le desired size
 - Normalmente seguito da Layout e Paint
- Layout
 - Usa le desired size già valide
 - Seguito da Paint
- Paint
 - Ridisegna soltanto
 - Non richiede Layout né Prepass

Fase	Cosa invalida
Prepass	Tutto (Size + Layout + Paint)
Layout	Layout + Paint
Paint	Solo rendering



Invalidation

Per evitare di ricalcolare layout, geometry e rendering ogni frame, i widget mantengono in cache la loro desired size e il risultato del layout.

A runtime, se qualcosa cambia (dimensione, contenuto, struttura), il widget viene invalidato per notificare a Slate che deve ricalcolare le informazioni necessarie.

Dentro la classe *InvalidateWidgetReason.h* trovate tutte le reason per cui un widget viene Invalidato



Invalidation

- **Paint:** cambia solo l'aspetto visivo (colore, testo, brush) → [LIGHT]
- **Layout:** cambia dimensione o disposizione → [HEAVY]
- **Prepass:** ricalcola tutte le desired size ricorsivamente → **[VERY HEAVY]**
- **Child Order:** aggiunta/rimozione figli → [HEAVY] (richiede Prepass + Layout)
- **Visibility:** cambio visibilità → [HEAVY] (spesso implica Layout)
- **Render Transform:** modifica transform (es. animazioni) → [MEDIUM]
- **Volatility:** widget marcato come dinamico (evita caching)



Visibility

La **Visibility** indica la visibilità e l'interattività di un widget.

- **Visible:** Il widget è renderizzato e interagibile tramite cursore. Usato principalmente per i button.
- **Collapsed:** Il widget non è renderizzato, non è interagibile e non occupa spazio nel layout
- **Hidden:** Il widget non è renderizzato, non è interagibile ma occupa spazio nel layout
- **Not Hit-Testable/Hit Test Invisible(Self & All Children):** Il widget è renderizzato, ma né il widget stesso né i child sono interagibili.
- **Not Hit-Testable/Hit Test Invisible(Self Only):** Il widget è renderizzato e solamente il widget stesso non è interagibile.

La **Visibility** inoltre viene utilizzata nel concetto di **Hit-Test Grid**, dove il valore **Visible** è quello meno "ottimizzato"



MINI SLATE BREAKDOWN



Slate Application

SlateApplication è il core system che gestisce Slate.

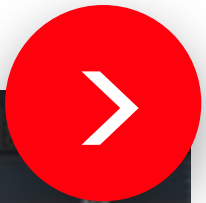
Gestisce:

- Input
- Focus
- Gestione finestre
- Rendering UI

Viene usato principalmente:

- In Tool di editor
- In Plugin di editor
- In custom window

```
FSlateApplication::Get().AddWindow(MyWindow);
```



Widget Name	FG	Visibility	Focus
SMultiBoxWidget		Visible	✓
SBorder		Visible	No
SVerticalBox		SelfHitTestInvisible	No
SScrollBox		Visible	Yes
SHorizontalBox		SelfHitTestInvisible	No
SOverlay		SelfHitTestInvisible	No
SScrollPanel		SelfHitTestInvisible	Yes
SVerticalBox		SelfHitTestInvisible	No
SHorizon		SelfHitTestInvisible	No
SHorizon		SelfHitTestInvisible	No
SHorizon		SelfHitTestInvisible	No
STextF		Collapsed	On Demand
SMenu		Visible	No
SM		Visible	✓
S		SelfHitTestInvisible	No
		SelfHitTestInvisible	No
		SelfHitTestInvisible	No
		Visible	No
		Visible	On Demand
		Visible	No
SHorizon		SelfHitTestInvisible	No
SHorizon		SelfHitTestInvisible	No

Widget Hierarchy

Il concetto di **Hierarchy** nei widget è creato usando i cosiddetti **Child Slot**.

Gli **Child Slot** sono oggetti opzionali che possono essere legati ai vari widget, poiché alcuni widget non sono progettati per avere child, come il widget “**Image**” che invece è un **LeafWidget**.

I widget sono solitamente di 3 tipi principali :

- **Leaf Widget**
- **Panel Widget**
- **Compound Widget**

Sono tutti figli di SWidget



SWidget

SWidget è la classe base di tutti i widget Slate.
Ogni elemento UI in Slate deriva da questa classe.

- STextBlock
- SButton
- SImage
- SEditableTextBox

Un SWidget gestisce:

- Rendering
- Input (mouse / tastiera)
- Layout
- Visibilità
- Tick / aggiornamento

In Slate tutti sono un SWidget.



SCompoundWidget

SCompoundWidget è un widget che può contenere altri widget. È la classe più usata quando si crea un widget custom.

Ha un singolo slot principale chiamato **ChildSlot**, dentro al quale si inserisce il layout.

SCompoundWidget va usato come classe base quando si vuole creare:

- Un widget custom
- Una UI composta da altri widget

```
class SMyWidget : public SCompoundWidget
{
public:

    SLATE_BEGIN_ARGS(SMyWidget) {}
    SLATE_END_ARGS()

    void Construct(const FArguments& InArgs);
};
```

```
void SMyWidget::Construct(const FArguments& InArgs)
{
    ChildSlot
    [
        SNew(STextBlock)
        .Text(FText::FromString("Hello Slate"))
    ];
}
```



SCompoundWidget





SPanelWidget

SPanelWidget è un widget che può avere figli e ha un numero di Child Slots dinamico. Gestisce il layout e la disposizione degli altri widget nella UI.

- SVerticalBox
- SHorizontal
- SOverlay
- SGridPanel
- SUniformGridPanel



SPanelWidget

Panel Widgets

Scroll Box

Text Block

Button w/Text

Text Block

Button w/Text

Text Block

Button w/Text

Vertical Box

Text Block

Button w/Text

Button w/Text

Button w/Text

Button w/Text

Horizontal Box

Button w/Text

Overlay



My cool message



SLeafWidget

SLeafWidget è un widget che non può avere figli.

È un nodo finale della UI.

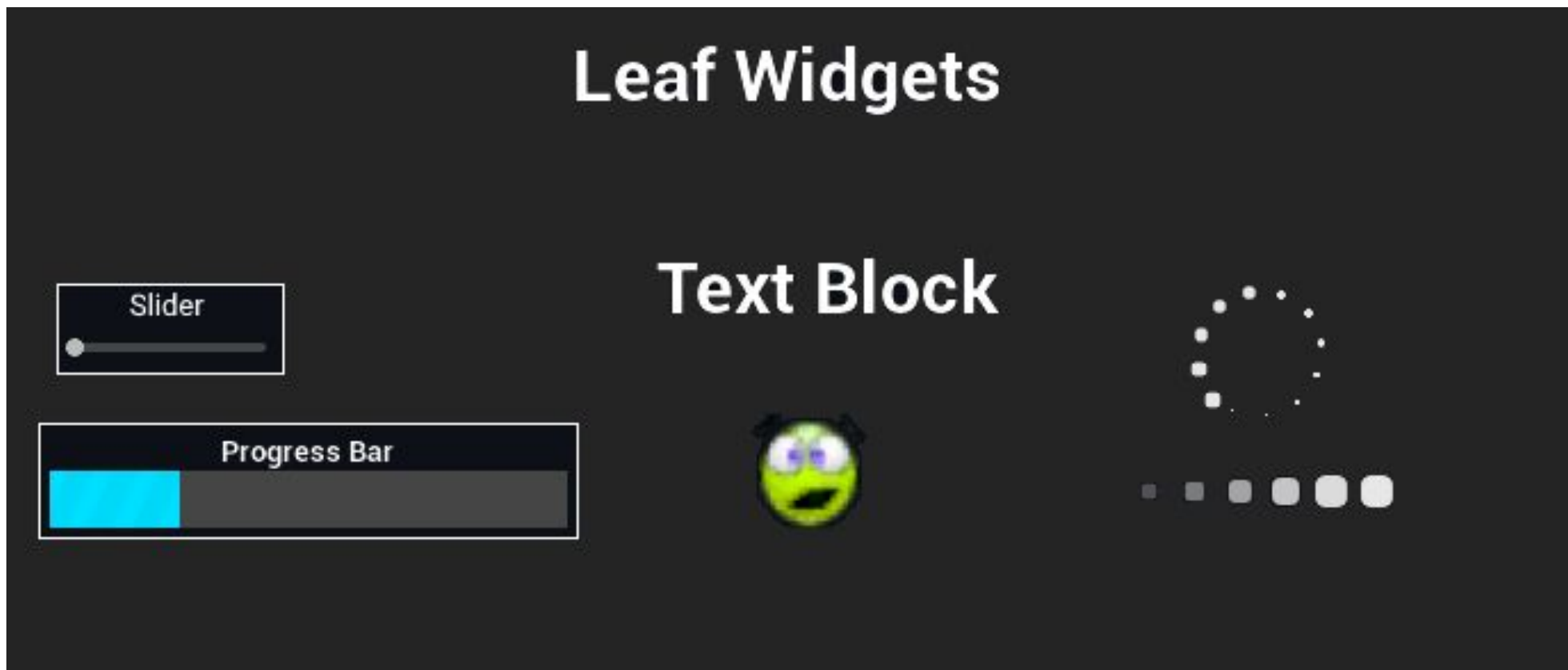
Un SLeafWidget è un widget che disegna qualcosa ma non contiene altri widget

- STextBlock
- SImage
- SSlider
- SProgressBar
- SSpacer
- SThrobber



Leaf Widgets

I **Leaf Widgets** sono semplicemente widgets senza Child Slots





Slate Macro

- **SLATE_BEGIN_ARGS / SLATE_END_ARGS**
Definiscono gli argomenti che un widget Slate può ricevere quando viene creato.
Sono equivalenti a parametri di costruzione.
- **SLATE_ARGUMENT**
Definisce un parametro semplice passato al widget.
È un valore statico.
- **SLATE_ATTRIBUTE**
Definisce un valore dinamico.
Il valore può essere aggiornato automaticamente tramite binding.
- **SLATE_EVENT**
Definisce un callback o evento.
Serve per collegare funzioni del codice agli eventi UI.
- **SLATE_NAMED_SLOT**
Definisce uno slot personalizzato per inserire widget figli.
Permette di creare layout più flessibili.



```
class SMyWidget : public SCompoundWidget
{
public:

    SLATE_BEGIN_ARGS(SMyWidget)
        : _Title(FText::FromString("Default Title"))
        , _Progress(0.0f)
    {}

    // parametro statico
    SLATE_ARGUMENT(FText, Title)

    // parametro dinamico
    SLATE_ATTRIBUTE(float, Progress)

    // evento
    SLATE_EVENT(FOnClicked, OnButtonClicked)

    // slot per widget figli
    SLATE_NAMED_SLOT(FArguments, Content)

    SLATE_END_ARGS()

public:

    void Construct(const FArguments& InArgs);

private:

    FText Title;
    TAttribute<float> Progress;
    FOnClicked OnButtonClicked;
};
```

```
void SMyWidget::Construct(const FArguments& InArgs)
{
    Title = InArgs._Title;
    Progress = InArgs._Progress;
    OnButtonClicked = InArgs._OnButtonClicked;

    ChildSlot
    [
        SNew(SVerticalBox)

        + SVerticalBox::Slot()
        .Padding(5)
        [
            SNew(STextBlock)
            .Text(Title)
        ]

        + SVerticalBox::Slot()
        .Padding(5)
        [
            SNew(SProgressBar)
            .Percent(Progress)
        ]

        + SVerticalBox::Slot()
        .Padding(5)
        [
            SNew(SButton)
            .Text(FText::FromString("Action"))
            .OnClicked(OnButtonClicked)
        ]

        + SVerticalBox::Slot()
        .Padding(5)
        [
            InArgs._Content.Widget
        ]
    ];
}
```



Slate Macro

- **SNew**

E' una macro utilizzata per creare un widget Slate.

Modo standard per istanziare i widget all'interno della sintassi dichiarativa di Slate.

- Crea un nuovo widget
- Inizializza gli argomenti dichiarati con le macro di Slate
- Restituisce uno **TSharedRef<SWidget>**

- **SAssignNew**

E' una macro simile a SNew, ma salva anche il riferimento del widget in una variabile. Serve quando esiste necessità di accedere al widget dopo la creazione.

TIPS

- **SNew** quando il widget serve solo nel layout.
- **SAssignNew** quando si deve riutilizzare il widget nel codice.

```
SNew(SMyWidget)
.Title(FText::FromString("Example Widget"))
.Progress(0.5f)
.OnButtonClicked(this, &SMYClass::HandleClick)
[
    SNew(STextBlock)
    .Text(FText::FromString("Custom Content"))
]
```




Domande?



